NAME: Radadiya Dhanashree B.
DIV : B
ROLL NO : H-10

.NET ASSIGNMENT-3

Q.1 Create a Blazor app to demonstrate collection data to user of your own choice

CODE :

model :-

```
namespace q1.Models
{
    public class Department
    {
        public int Id { get; set; }

        public string DepartmentName { get; set; } = "";

        public string HOD { get; set; } = "";

        public string Course { get; set; } = "";

        public int NoOfStudents { get; set; }
    }
}
```

page:-

```
@page "/department"
@using q1.Models

<PageTitle>Department</PageTitle>

<h1>Department Management</h1>

<div class="form-box">

    <h3>Add Department</h3>

    <div class="form-group">
        <label>Department Name</label>
        <input @bind="newDepartment.DepartmentName" />
    </div>

    <div class="form-group">
        <label>HOD</label>
        <input @bind="newDepartment.HOD" />
    </div>

    <div class="form-group">
        <label>Course</label>
        <input @bind="newDepartment.Course" />
    </div>

    <div class="form-group">
        <label>No. of Students</label>
        <input type="number" @bind="newDepartment.NoOfStudents" />
    </div>

    <button @onclick="AddDepartment">
```

```
        Add Department
    </button>

</div>

<hr />

<h3>Department List</h3>

<table>
    <thead>
        <tr>
            <th>Id</th>
            <th>Department Name</th>
            <th>HOD</th>
            <th>Course</th>
            <th>No. of Students</th>
        </tr>
    </thead>

    <tbody>
        @foreach (var department in departments)
        {
            <tr>
                <td>@department.Id</td>
                <td>@department.DepartmentName</td>
                <td>@department.HOD</td>
                <td>@department.Course</td>
                <td>@department.NoOfStudents</td>
            </tr>
        }
    </tbody>
</table>

<style>

    .form-box {
        width: 500px;
        padding: 20px;
        border: 1px solid #ccc;
        border-radius: 8px;
        margin-bottom: 20px;
    }

    .form-group {
        margin-bottom: 15px;
    }

    label {
        display: block;
        margin-bottom: 5px;
        font-weight: bold;
    }

    input {
        width: 100%;
        padding: 8px;
        border: 1px solid #aaa;
        border-radius: 4px;
    }

    button {
        padding: 10px 20px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

        button:hover {
```

```
            background-color: #0056b3;
        }

    table {
        width: 100%;
        border-collapse: collapse;
    }

    th,
    td {
        border: 1px solid #ccc;
        padding: 10px;
        text-align: left;
    }

    th {
        background-color: #f2f2f2;
    }

</style>

@code {

    private List<Department> departments = new List<Department>
    {
        new Department
        {
            Id = 1,
            DepartmentName = "Computer Engineering",
            HOD = "Dr. Patel",
            Course = "B.Tech",
            NoOfStudents = 120
        },

        new Department
        {
            Id = 2,
            DepartmentName = "Information Technology",
            HOD = "Dr. Shah",
            Course = "B.Tech",
            NoOfStudents = 100
        },

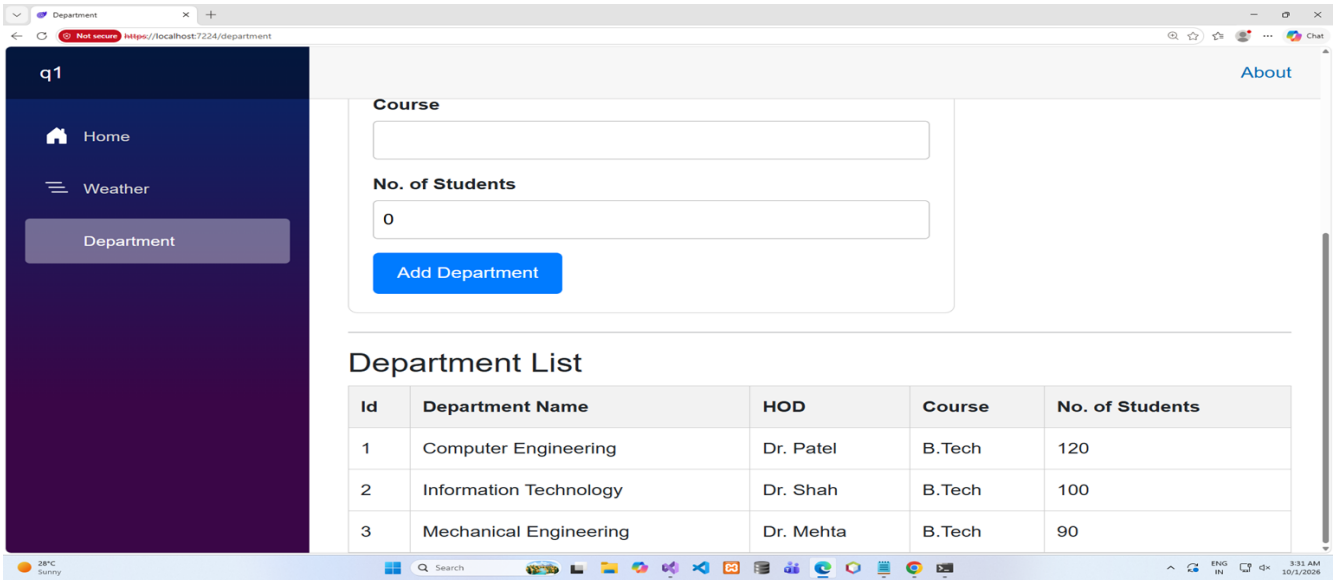
        new Department
        {
            Id = 3,
            DepartmentName = "Mechanical Engineering",
            HOD = "Dr. Mehta",
            Course = "B.Tech",
            NoOfStudents = 90
        }
    };

    private Department newDepartment = new Department();

    private void AddDepartment()
    {
        newDepartment.Id = departments.Count + 1;

        departments.Add(newDepartment);

        newDepartment = new Department();
    }
}
```



Q.2 Create a Blazor app with components and create without parameter and with para components to show complex object collection

CODE :

model:-

```
namespace q2.Models
{
    public class Department
    {
        public int id { get; set; }

        public string name { get; set; } = "";

        public string location { get; set; } = "";

        public string manager { get; set; } = "";

        public int employees { get; set; }
    }
}
```

page:-

DepartmentCard

```
@using q2.Models

<div class="department-card">
    <h3>@department.name</h3>

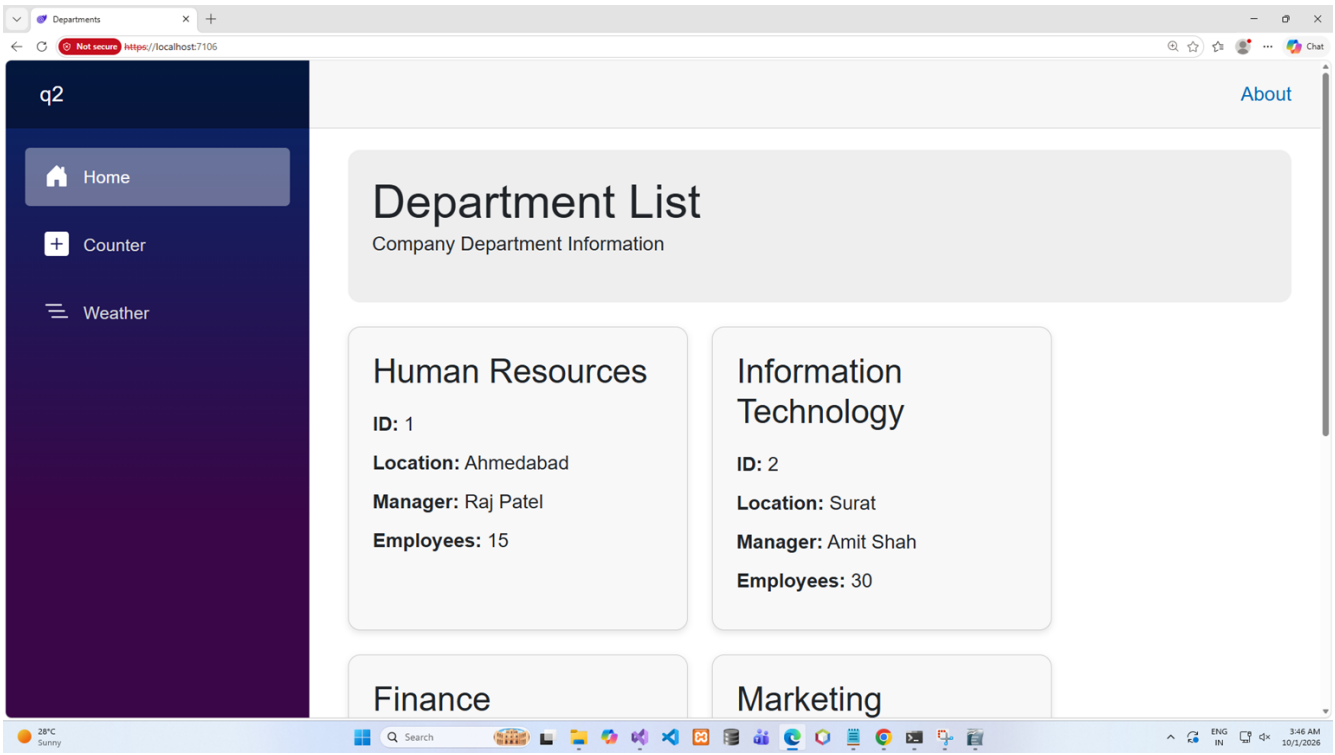
    <p><b>ID:</b> @department.id</p>
    <p><b>Location:</b> @department.location</p>
    <p><b>Manager:</b> @department.manager</p>
    <p><b>Employees:</b> @department.employees</p>
</div>

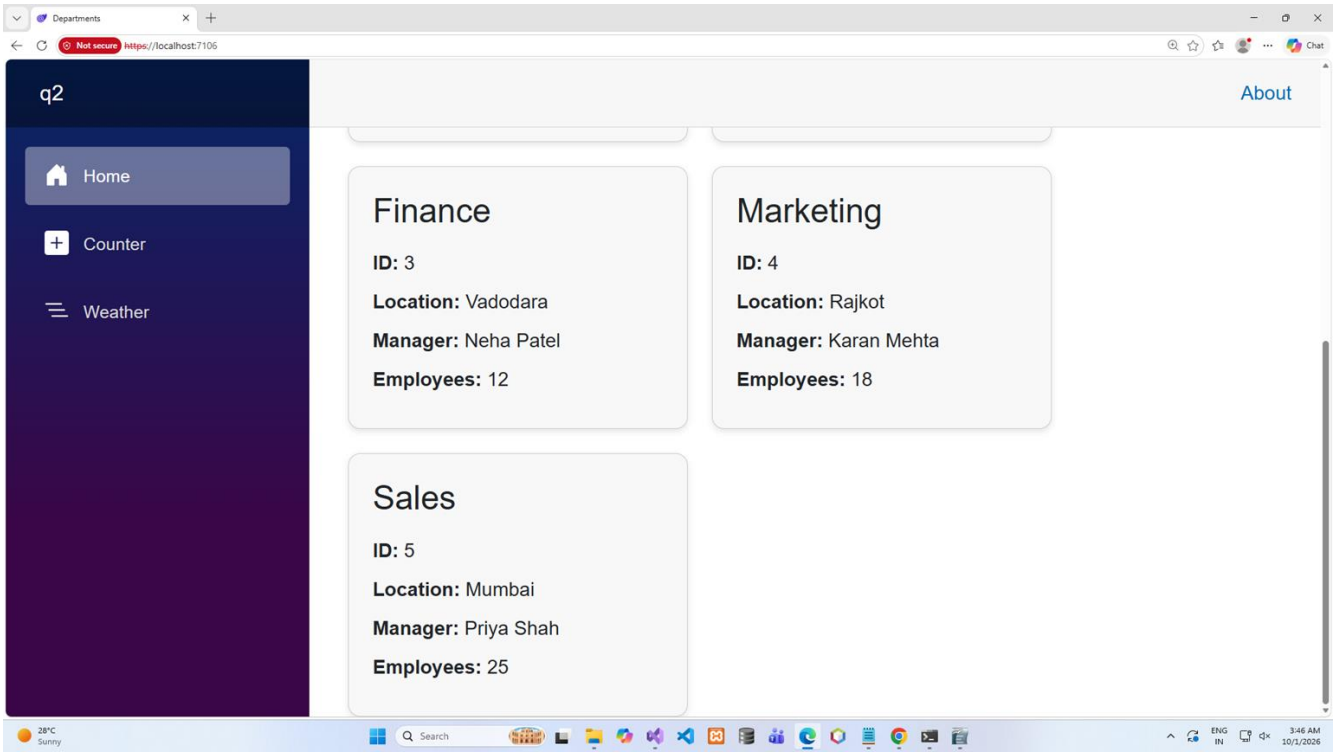
@code {
    [Parameter]
```

```
    public Department department { get; set; } = new Department();
}
```

DepartmentHeader

```
    <div class="department-header">
    <h1>Department List</h1>
    <p>Company Department Information</p>
</div>
```





Q.3 Create a Blazor app to show multiple components based dashboard to users

CODE:

Page:-

Counter

```
@page "/counter"
@rendermode InteractiveServer

<PageTitle>Counter</PageTitle>

<h1>Counter</h1>

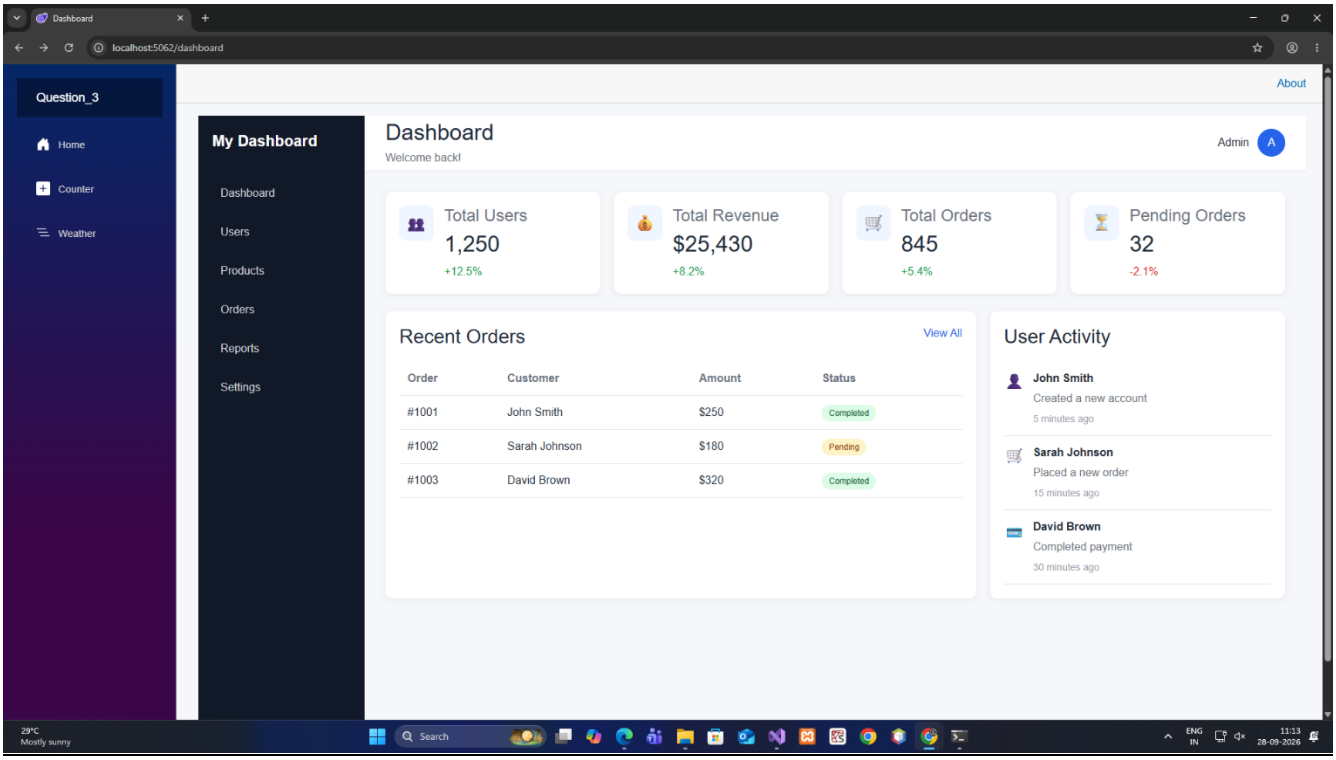
<p role="status">Current count: @currentCount</p>

<button class="btn btn-primary" @onclick="IncrementCount">Click me</button>

@code {
    private int currentCount = 0;

    private void IncrementCount()
```

```
    {
        currentCount++;
    }
}
```



Q.4 Create an interactive Blazor components to take user input and display all user's inputs in the collection

CODE:

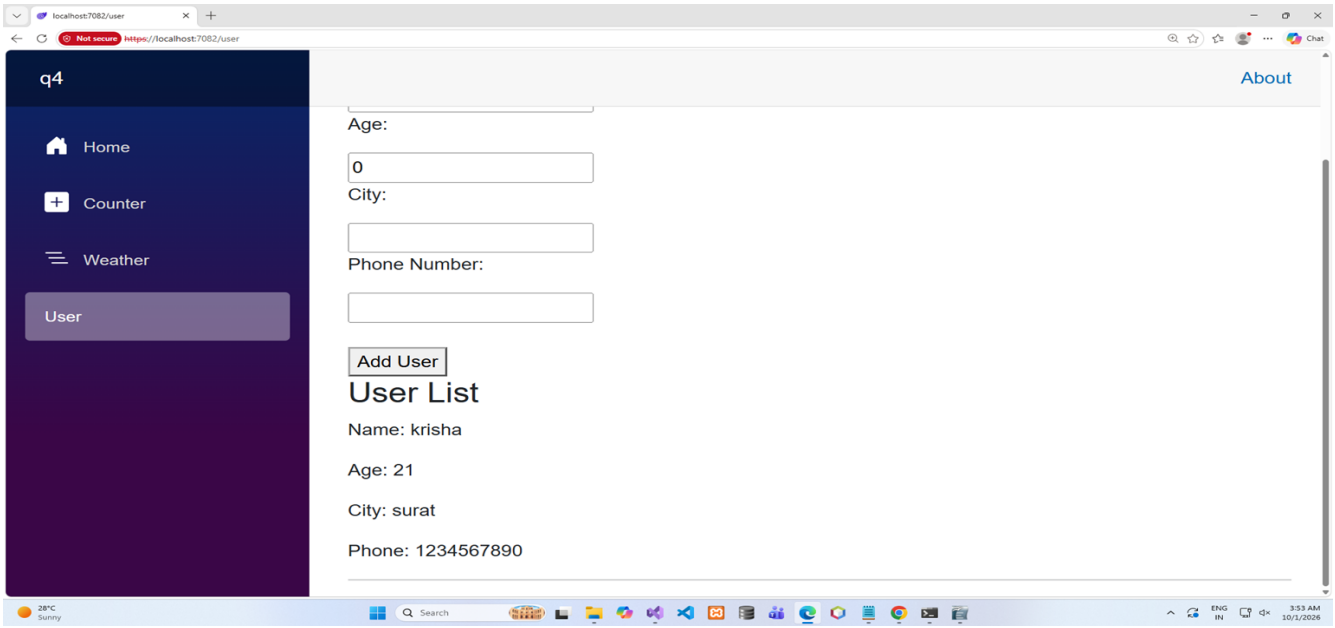
model:-

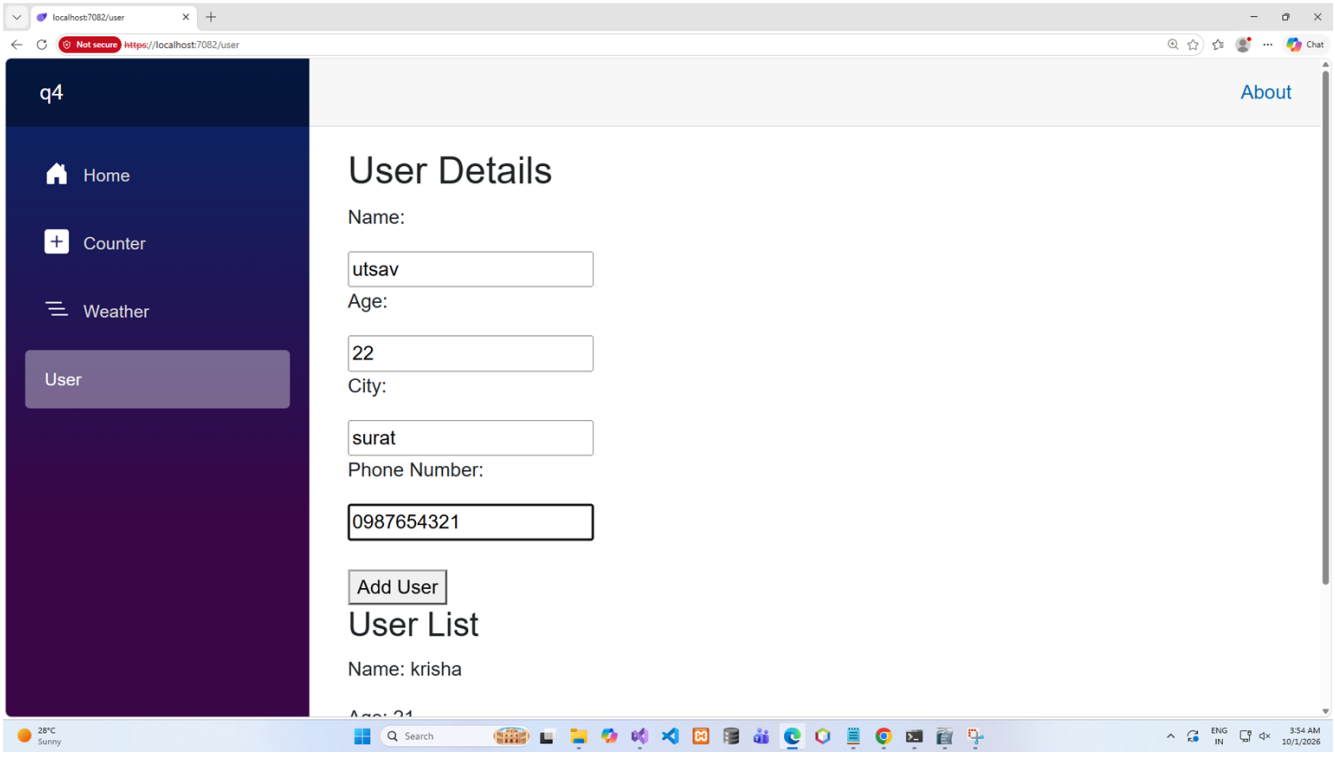
```
namespace q4.Models
{
    public class User
    {
        public string name { get; set; } = "";
        public int age { get; set; }
        public string city { get; set; } = "";
        public string phone { get; set; } = "";
    }
}
```

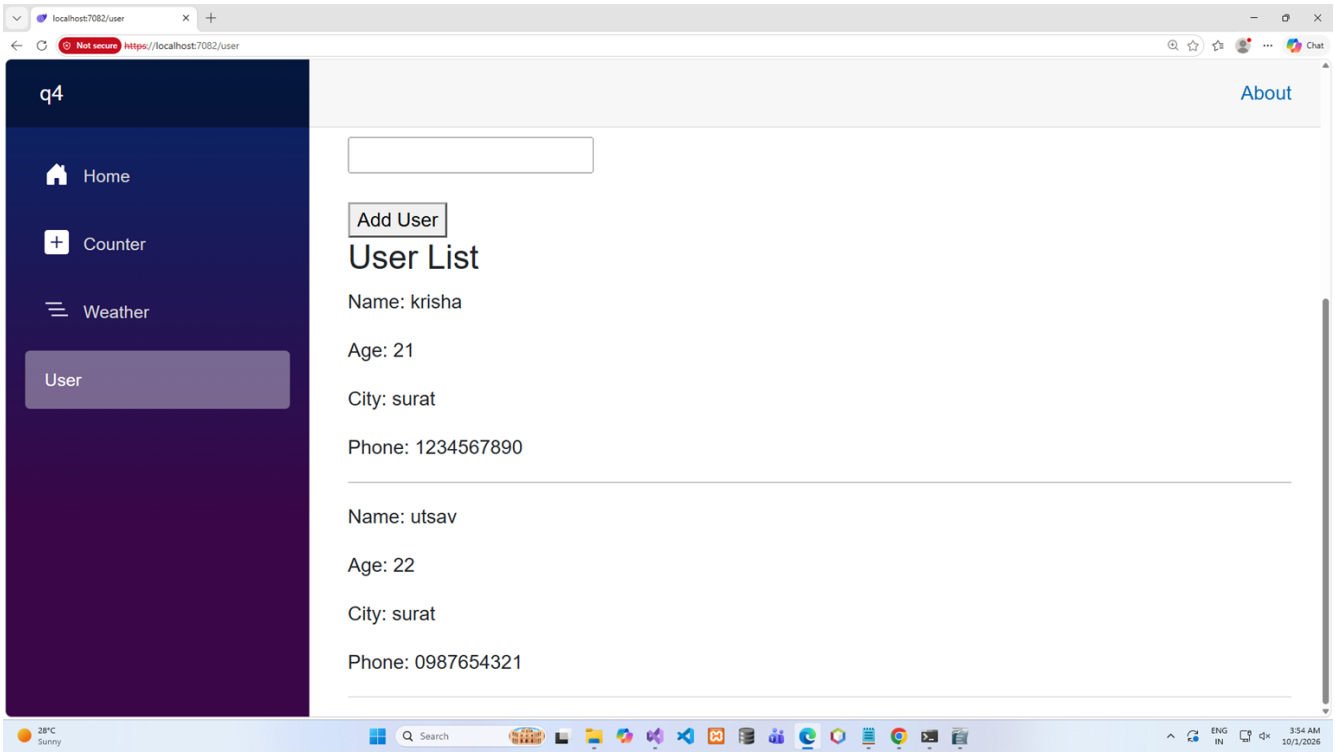
page:-

user

```
@page "/user"
@rendermode InteractiveServer
<UserInput />
```


localhost:7082/user
Not secure https://localhost:7082/user
q4
About
Home
Counter
Weather
User
Age:
0
City:
Phone Number:
Add User
User List
Name: krisha
Age: 21
City: surat
Phone: 1234567890


localhost:7082/user
Not secure https://localhost:7082/user
q4
About
Home
Counter
Weather
User
User Details
Name:
utsav
Age:
22
City:
surat
Phone Number:
0987654321
Add User
User List
Name: krisha



Q.5 Create a Web API to calculate a Simple Interest based on PrincipalAmount, RateOfInterest and NoOfYears provided by MVC Core Web Application. Provide proper user interface of MVC Web Application to input all values and show all results to the users after calculating Simple Interest using Web API. List of Result must have all information as PrincipalAmount, RateOfInterest, NoOfYears, SimpleInterest and TotalAmount. (TotalAmout should be calculated using C# Property type as PrincipalAmount + SimpleInterest)

CODE:

model:-

SimpleInterest.cs

```
namespace q5.Models
{
    public class SimpleInterest
    {
        public double PrincipalAmount { get; set; }
        public double RateOfInterest { get; set; }
        public int NoOfYears { get; set; }
        public double SimpleInterestValue { get; set; }
        public double TotalAmount
        {
```

```
            get
            {
                return PrincipalAmount + SimpleInterestValue;
            }
        }
    }
}
```

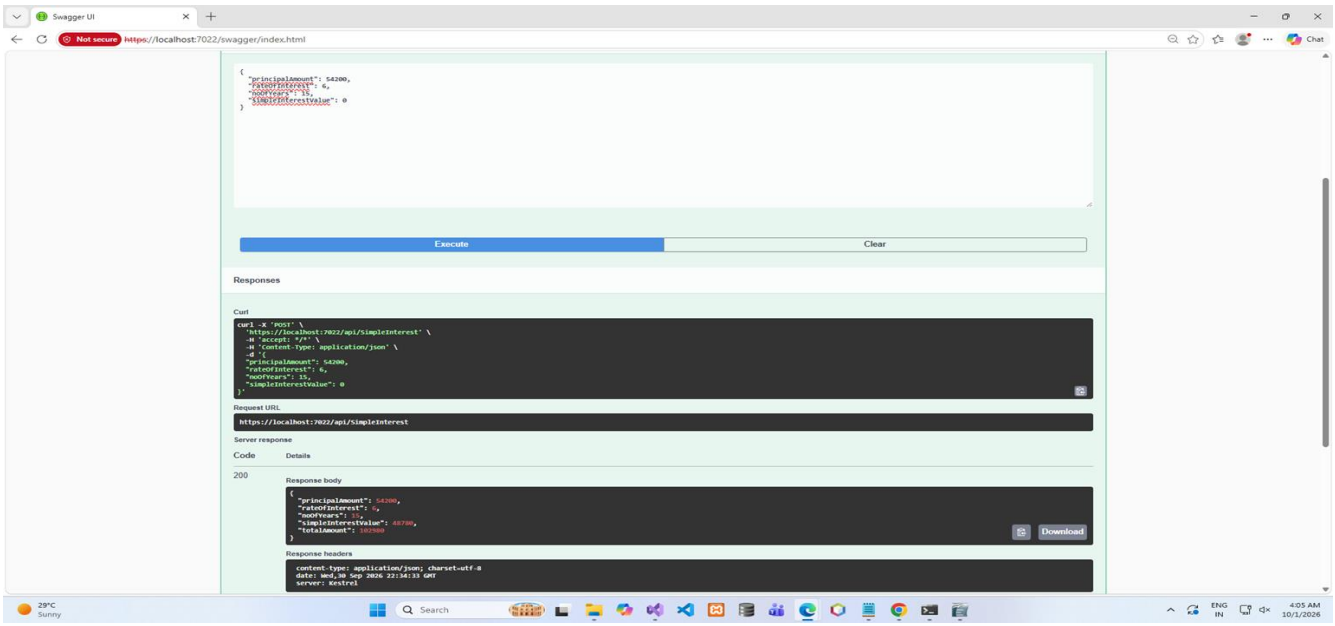
controller:-

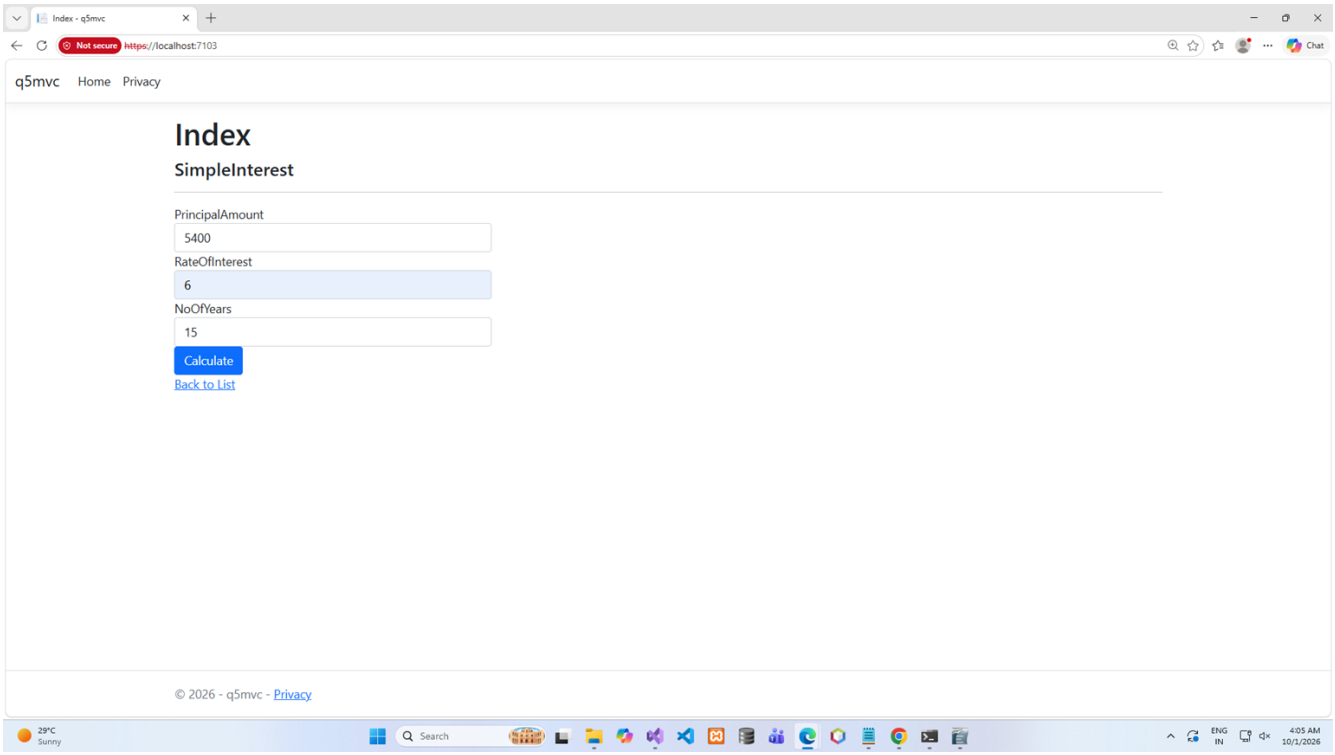
SimpleInterestController.cs

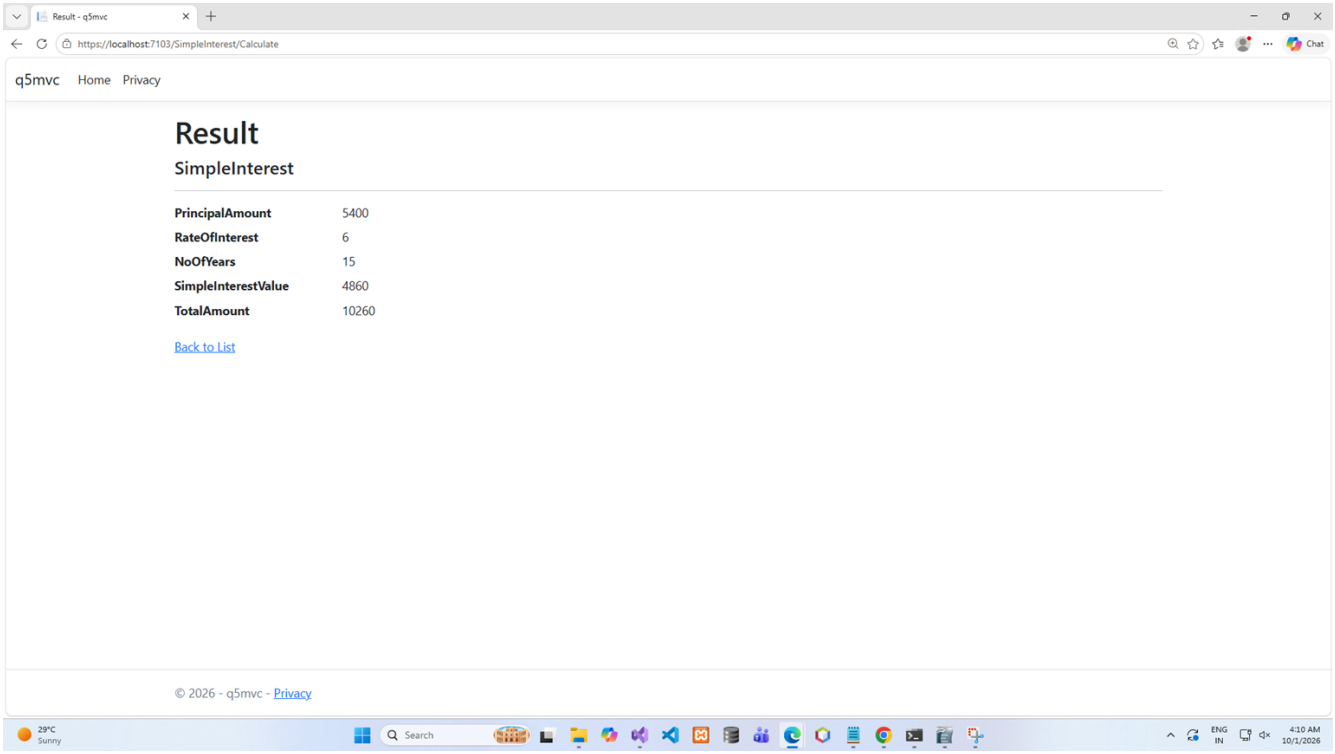
```
using Microsoft.AspNetCore.Mvc;
using q5.Models;

namespace q5.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class SimpleInterestController : ControllerBase
    {
        [HttpPost]
        public IActionResult Calculate(SimpleInterest data)
        {
            data.SimpleInterestValue =
                data.PrincipalAmount *
                data.RateOfInterest *
                data.NoOfYears / 100;

            return Ok(data);
        }
    }
}
```







Q.6 Create a simple Blazor Web App to calculate Simple Interest as per user Input of PrincipalAmount, RateOfInterest and NoOfYears. Create a function to calculate Interest and use this function to show Interest Amount as soon as user insert PrincipalAmount, RateOfInterest and NoOfYears. Provide Add button to add calculated data to the Generic List and show all records to the user.

CODE:

Model:-

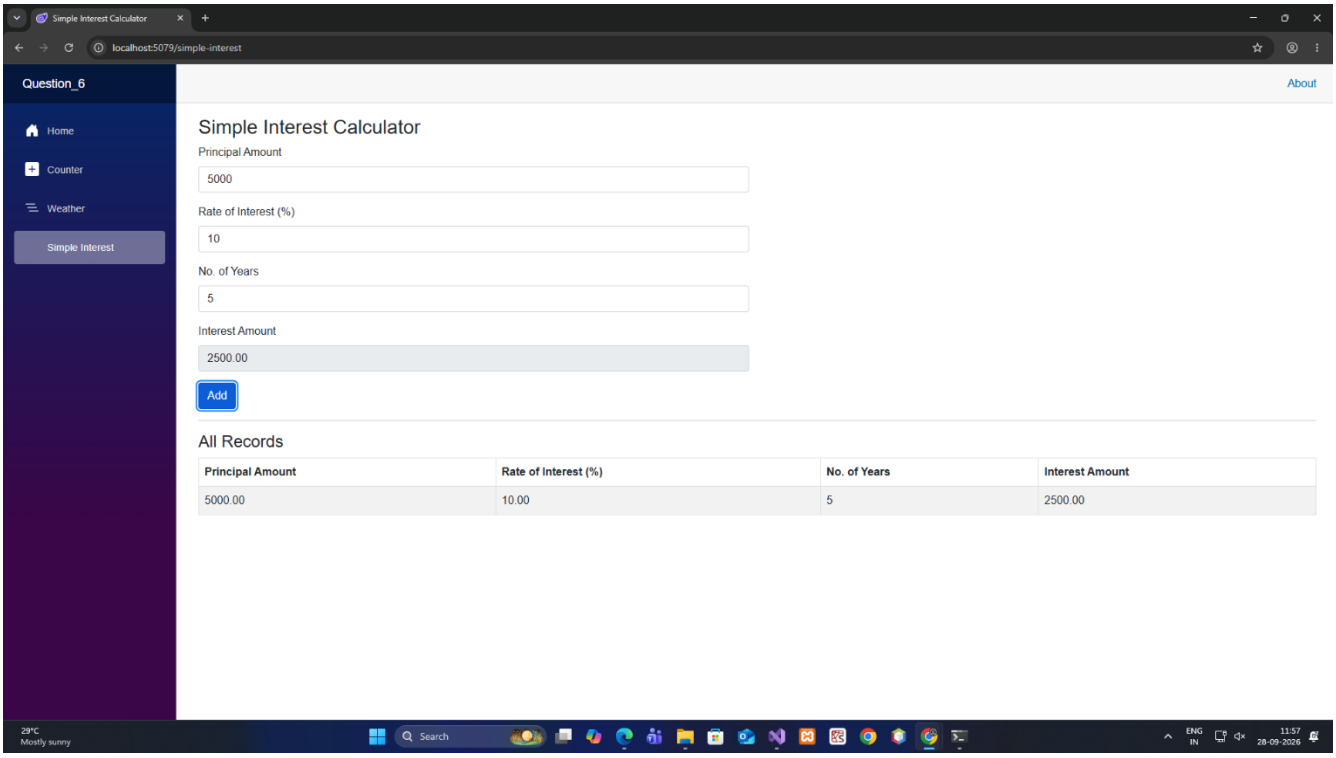
SimpleInterest

```
namespace q6.Models
{
    public class SimpleInterest
    {
        public double principalamount { get; set; }

        public double rateofinterest { get; set; }

        public int noofyears { get; set; }

        public double interestamount { get; set; }
    }
}
```



Q.7 Create a Minimal API for Book Information and consume it in a Blazor application to perform CRUD operations.

CODE:

Model:-

Books

```
namespace q7.Models
{
    public class Book
    {
        public int Id { get; set; }

        public string Title { get; set; }

        public string Author { get; set; }

        public double Price { get; set; }
    }
}
```

Page:-

Book razor

```
@page "/books"
@rendermode InteractiveServer

@using q7Blazor.Models
@inject IHttpClientFactory HttpClientFactory

<h3>Book Management</h3>

<h4>Add New Book</h4>

<div class="mb-3">
    <label>Title</label>
    <input class="form-control" @bind="newBook.Title" />
</div>

<div class="mb-3">
    <label>Author</label>
    <input class="form-control" @bind="newBook.Author" />
</div>

<div class="mb-3">
    <label>Price</label>
    <input class="form-control" @bind="newBook.Price" />
</div>

<button class="btn btn-primary" @onclick="AddBook">
    Add Book
</button>
```

```
<hr />

@if (isEditing)
{
    <h4>Edit Book</h4>

    <div class="mb-3">
        <label>Title</label>
        <input class="form-control" @bind="editBook.Title" />
    </div>

    <div class="mb-3">
        <label>Author</label>
        <input class="form-control" @bind="editBook.Author" />
    </div>

    <div class="mb-3">
        <label>Price</label>
        <input class="form-control" @bind="editBook.Price" />
    </div>

    <button class="btn btn-success" @onclick="UpdateBook">
        Update Book
    </button>

    <button class="btn btn-secondary ms-2" @onclick="CancelEdit">
        Cancel
    </button>

    <hr />
}

@if (books == null)
{
    <p>Loading books...</p>
}
else
{
    <table class="table table-bordered">

        <thead>
            <tr>
                <th>Id</th>
                <th>Title</th>
                <th>Author</th>
                <th>Price</th>
                <th>Action</th>
            </tr>
```

```
        </thead>

        <tbody>

            @foreach (var book in books)
            {
                <tr>
                    <td>@book.Id</td>
                    <td>@book.Title</td>
                    <td>@book.Author</td>
                    <td>@book.Price</td>

                    <td>
                        <button class="btn btn-warning"
                                @onclick="() => EditBook(book)">
                            Edit
                        </button>

                        <button class="btn btn-danger ms-2"
                                @onclick="() => DeleteBook(book.Id)">
                            Delete
                        </button>
                    </td>
                </tr>
            }

        </tbody>

    </table>
}

@code {

    private List<Book> books = new();

    private Book newBook = new();

    private Book editBook = new();

    private bool isEditing = false;

    protected override async Task OnInitializedAsync()
    {
        var client = HttpClientFactory.CreateClient("BookAPI");

        books = await client.GetFromJsonAsync<List<Book>>(
            "api/books"
        ) ?? new List<Book>();
    }
```

```
// CREATE
private async Task AddBook()
{
    var client = HttpClientFactory.CreateClient("BookAPI");

    var response = await client.PostAsJsonAsync(
        "api/books",
        newBook
    );

    if (response.IsSuccessStatusCode)
    {
        var book = await response.Content.ReadFromJsonAsync<Book>();

        if (book != null)
        {
            books.Add(book);
        }

        newBook = new Book();
    }
}

// EDIT
private void EditBook(Book book)
{
    editBook = new Book
        {
            Id = book.Id,
            Title = book.Title,
            Author = book.Author,
            Price = book.Price
        };

    isEditing = true;
}

// UPDATE
private async Task UpdateBook()
{
    var client = HttpClientFactory.CreateClient("BookAPI");

    var response = await client.PutAsJsonAsync(
        $"api/books/{editBook.Id}",
        editBook
    );

    if (response.IsSuccessStatusCode)
```

```
    {
        var book = await response.Content.ReadFromJsonAsync<Book>();

        if (book != null)
        {
            var oldBook = books.FirstOrDefault(
                b => b.Id == book.Id
            );

            if (oldBook != null)
            {
                oldBook.Title = book.Title;
                oldBook.Author = book.Author;
                oldBook.Price = book.Price;
            }
        }

        isEditing = false;
        editBook = new Book();
    }
}

// CANCEL EDIT
private void CancelEdit()
{
    isEditing = false;
    editBook = new Book();
}

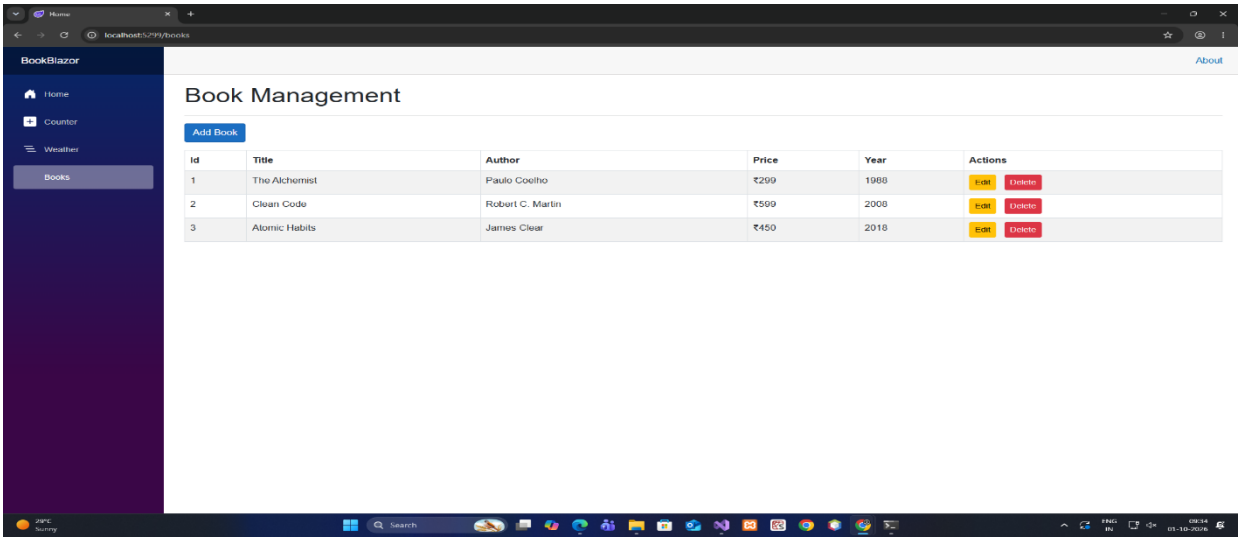
// DELETE
private async Task DeleteBook(int id)
{
    var client = HttpClientFactory.CreateClient("BookAPI");

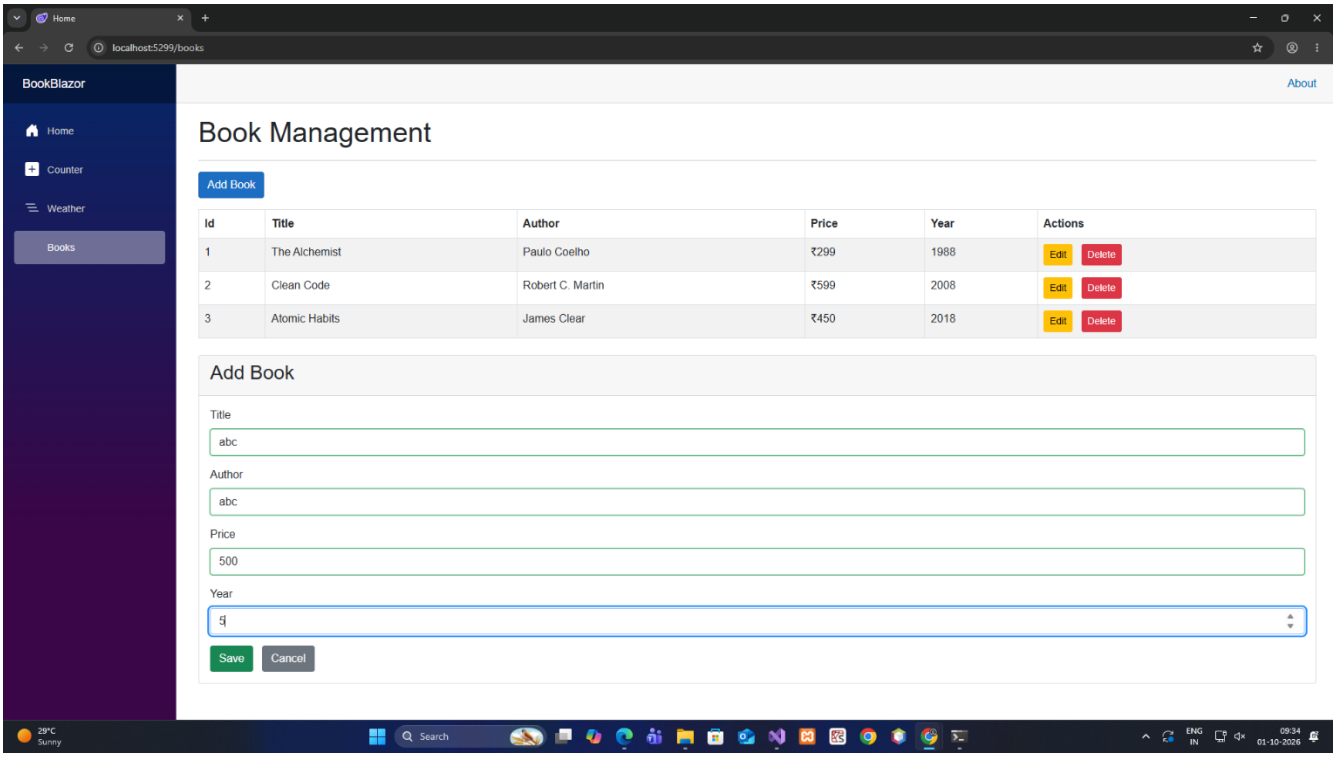
    var response = await client.DeleteAsync(
        $"api/books/{id}"
    );

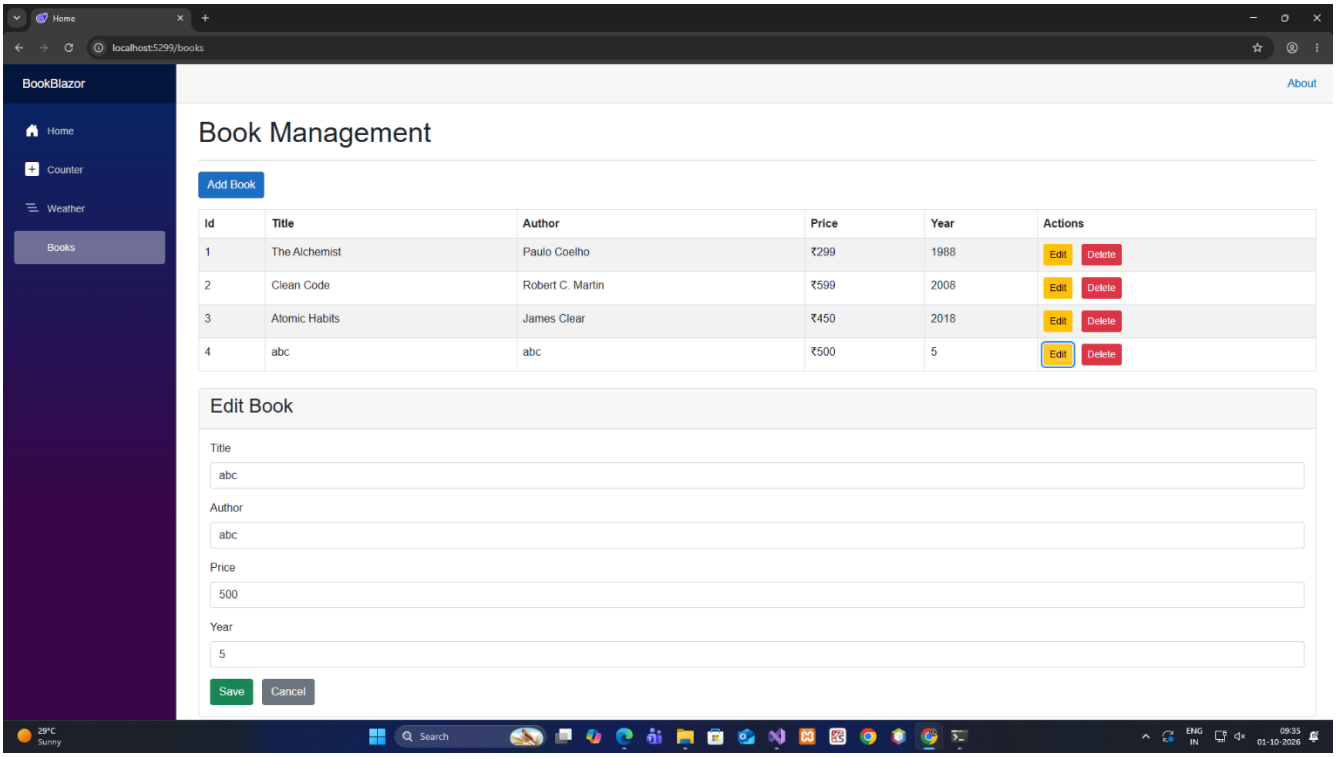
    if (response.IsSuccessStatusCode)
    {
        var book = books.FirstOrDefault(
            b => b.Id == id
        );

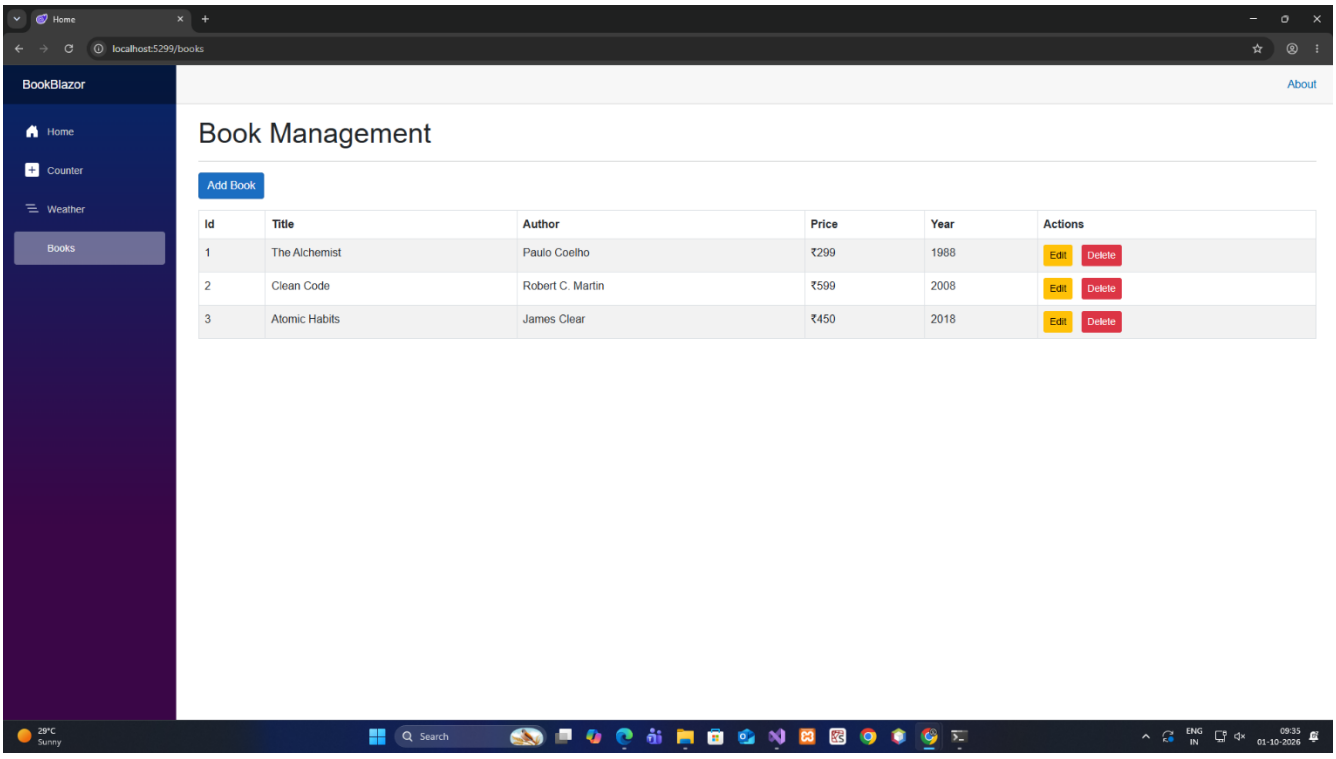
        if (book != null)
        {
            books.Remove(book);
        }
    }
```

```
    }
}
```


BookBlazor
About
Home
Counter
Weather
Books
Book Management
Add Book
Id
Title
Author
Price
Year
Actions
1
The Alchemist
Paulo Coelho
₹299
1988
Edit
Delete
2
Clean Code
Robert C. Martin
₹599
2008
Edit
Delete
3
Atomic Habits
James Clear
₹450
2018
Edit
Delete


Home
localhost:5299/books
BookBlazor
About
Home
Counter
Weather
Books
Book Management
Add Book
Id
Title
Author
Price
Year
Actions
1
The Alchemist
Paulo Coelho
₹299
1988
Edit
Delete
2
Clean Code
Robert C. Martin
₹599
2008
Edit
Delete
3
Atomic Habits
James Clear
₹450
2018
Edit
Delete
Add Book
Title
abc
Author
abc
Price
500
Year
5
Save
Cancel
29°C
Sunny
Search
ENG
IN
09:34
01-10-2026





Q.8 Create a Blazor web application to consume a Web API to perform CRUD operations on your project related tables. (Select any two tables 1 Primary and 1 Secondary table)

CODE:

Controller:-

DepartmentController

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using q8.Data;
using q8.Models;

namespace q8.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class DepartmentController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public DepartmentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/Department
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Department>>> GetDepartments()
        {
            return await _context.Departments.ToListAsync();
        }

        // GET: api/Department/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Department>> GetDepartment(int id)
        {
            var department = await _context.Departments.FindAsync(id);

            if (department == null)
            {
                return NotFound();
            }

            return department;
        }

        // POST: api/Department
        [HttpPost]
        public async Task<ActionResult<Department>> PostDepartment(
            Department department)
        {
```

```
            _context.Departments.Add(department);

            await _context.SaveChangesAsync();

            return CreatedAtAction(
                nameof(GetDepartment),
                new { id = department.Id },
                department
            );
        }

        // PUT: api/Department/5
        [HttpPut("{id}")]
        public async Task<IActionResult> PutDepartment(
            int id,
            Department department)
        {
            if (id != department.Id)
            {
                return BadRequest();
            }

            _context.Entry(department).State = EntityState.Modified;

            await _context.SaveChangesAsync();

            return NoContent();
        }

        // DELETE: api/Department/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteDepartment(int id)
        {
            var department = await _context.Departments.FindAsync(id);

            if (department == null)
            {
                return NotFound();
            }

            _context.Departments.Remove(department);

            await _context.SaveChangesAsync();

            return NoContent();
        }
    }
}
```

FacultyController

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using q8.Data;
using q8.Models;

namespace q8.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class FacultyController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public FacultyController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/Faculty
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Faculty>>> GetFaculties()
        {
            return await _context.Faculties.ToListAsync();
        }

        // GET: api/Faculty/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Faculty>> GetFaculty(int id)
        {
            var faculty = await _context.Faculties.FindAsync(id);

            if (faculty == null)
            {
                return NotFound();
            }

            return faculty;
        }

        // POST: api/Faculty
        [HttpPost]
        public async Task<ActionResult<Faculty>> PostFaculty(
            Faculty faculty)
        {
            _context.Faculties.Add(faculty);
```

```
            await _context.SaveChangesAsync();

            return CreatedAtAction(
                nameof(GetFaculty),
                new { id = faculty.Id },
                faculty
            );
        }

        // PUT: api/Faculty/5
        [HttpPut("{id}")]
        public async Task<IActionResult> PutFaculty(
            int id,
            Faculty faculty)
        {
            if (id != faculty.Id)
            {
                return BadRequest();
            }

            _context.Entry(faculty).State = EntityState.Modified;

            await _context.SaveChangesAsync();

            return NoContent();
        }

        // DELETE: api/Faculty/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteFaculty(int id)
        {
            var faculty = await _context.Faculties.FindAsync(id);

            if (faculty == null)
            {
                return NotFound();
            }

            _context.Faculties.Remove(faculty);

            await _context.SaveChangesAsync();

            return NoContent();
        }
    }
}
```

Model:-

Department

```
namespace q8.Models
{
    public class Department
    {
        public int Id { get; set; }

        public string Name { get; set; }
    }
}
```

Faculty:-

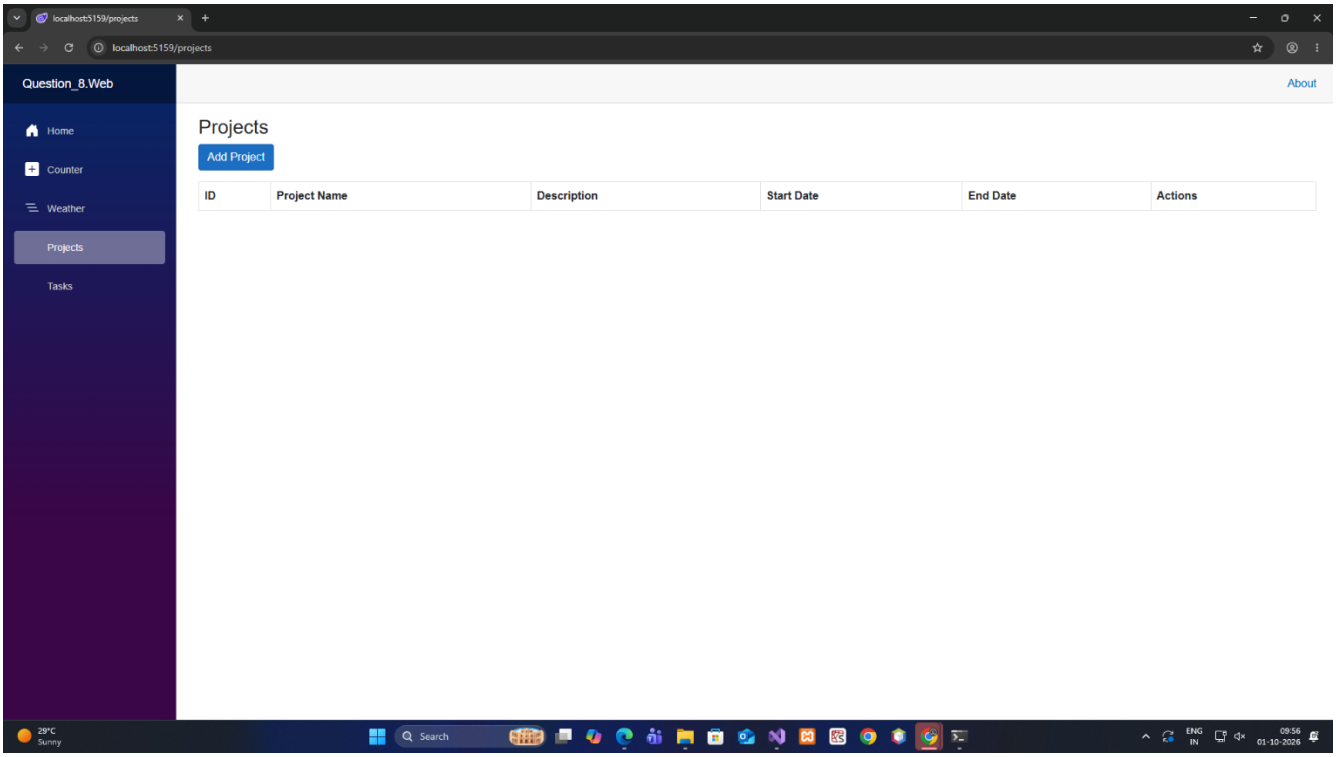
```
namespace q8.Models
{
    public class Faculty
    {
        public int Id { get; set; }

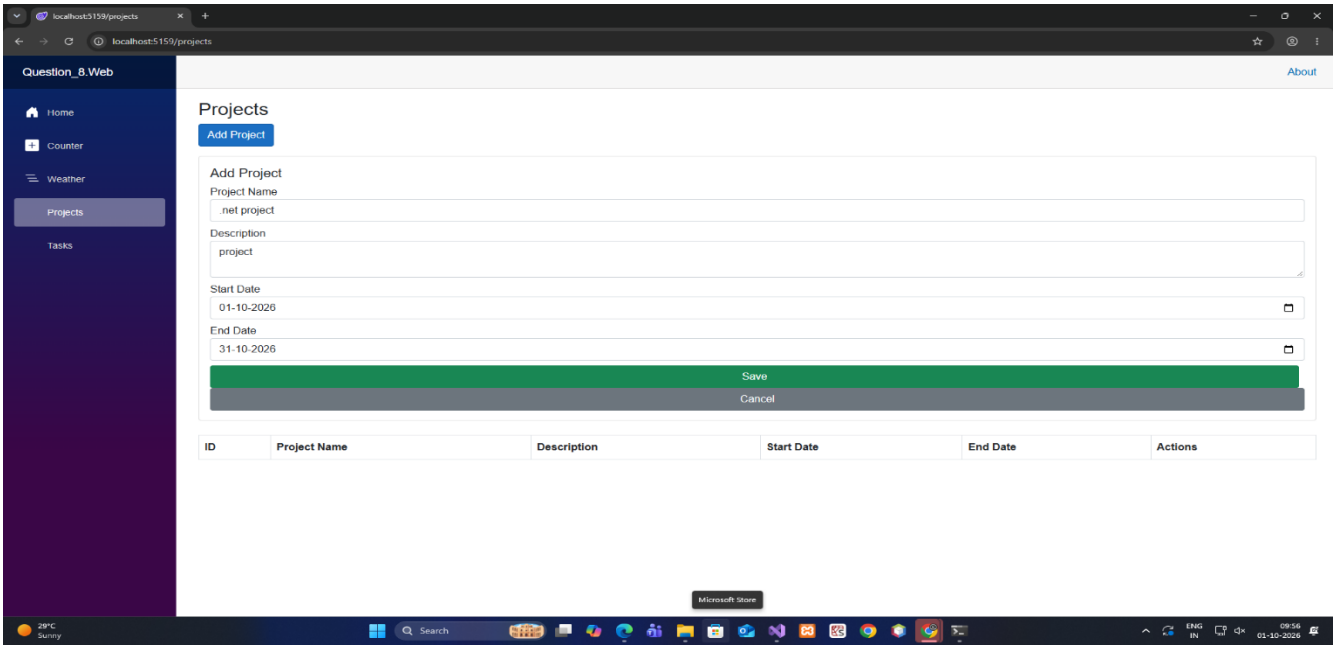
        public string Name { get; set; }

        public string Email { get; set; }

        public string Subject { get; set; }

        public int DepartmentId { get; set; }
    }
}
```


localhost:5159/projects
Question_8.Web
About
Home
Counter
Weather
Projects
Tasks
Projects
Add Project
ID
Project Name
Description
Start Date
End Date
Actions


localhost:5159/projects
Question_8.Web
About
Home
Counter
Weather
Projects
Tasks
Projects
Add Project
Add Project
Project Name
.net project
Description
project
Start Date
01-10-2026
End Date
31-10-2026
Save
Cancel
ID
Project Name
Description
Start Date
End Date
Actions
Microsoft Store